

C++ Programming: Systems and Programming Concepts

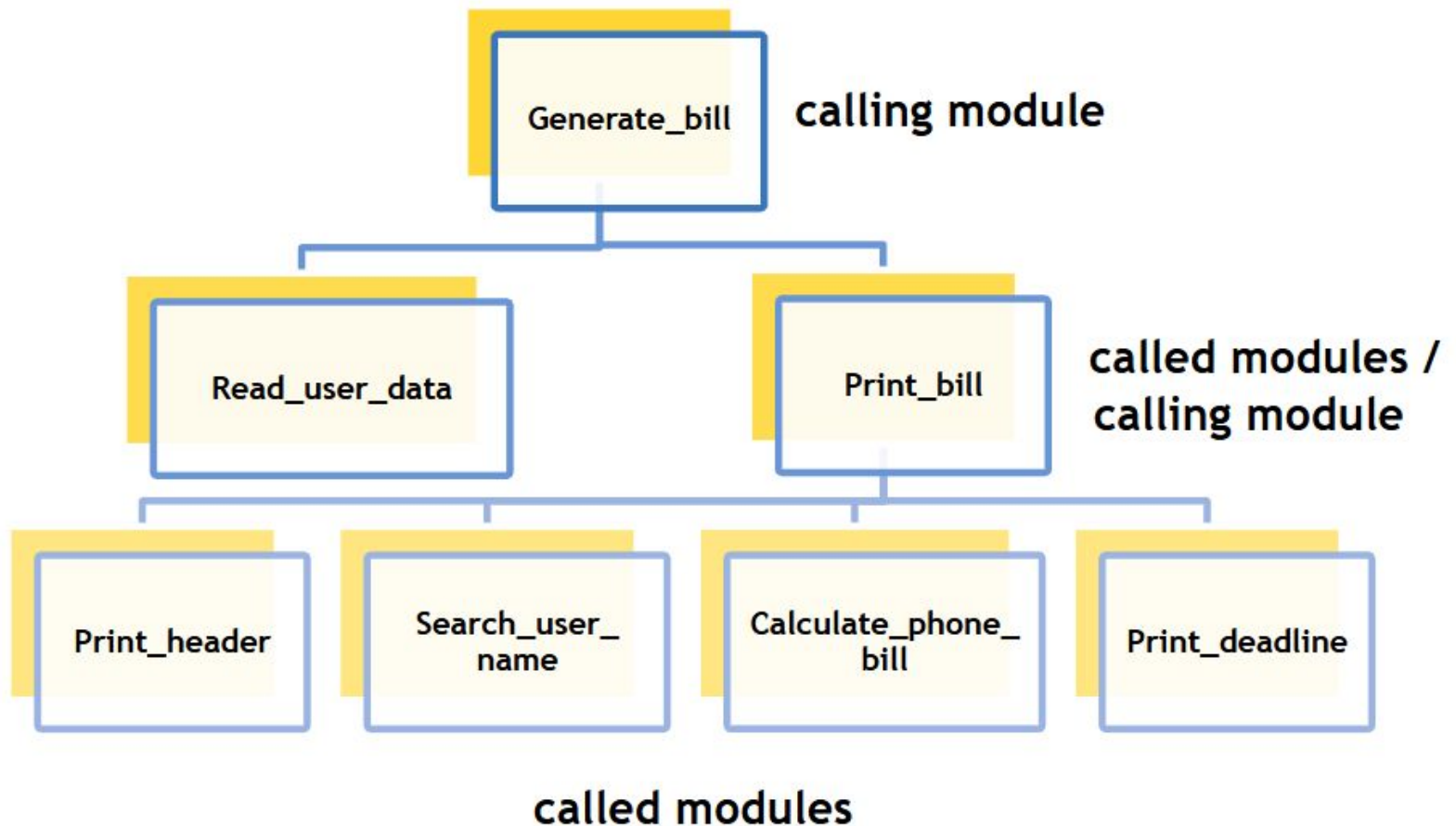
Chapter 5 Advanced Functions I

Why Function? - Recap

- **Modular programming** enables us to organize a program into small, independent modules / functions that are separately named & individually invoke-able program elements.
- Each module performs separate task with a single purpose.
- Benefits:- more manageable codes, codes reusability, data protection, ease of debugging.
- Structure chart is used to show the hierarchical relationship among all the modules.

Why Function? - Recap

- Structure chart - Example



Why Function? - Recap

- Sample of function:

```
void header();           // 1. Function declaration

void main() {
    header();             // 2. Function call
}

void header() {           // 3. Function definition
    cout << "function";
}
```

Why Function? - Recap

- Types of function:

1. *Void* function

- Does NOT return a function value.

Can receive value from the calling function

2. *Value-returning* function

- Will return the function value to the calling module/function by using *return* statement

Functions with parameters

- ❑ To advance in C++, understanding of the various ways to pass parameters to / from the functions is essential for efficiency, clarity and data protection.
- ❑ Global variables
 - Declared outside any functions and can be accessed by all functions in a program
 - Data protection is compromised, data can be accessed and changed by anyone anytime
 - Dangerous if use and so not recommended !

Functions with parameters

- Use of functions with parameters to ensure
 - Data protection – passing of data via parameters enable functions to operate on its own copy and better control of shared data.
 - Modularity – functions will be self-contained and independent, improve codes organization, ease of managing, testing, reading and debugging.
 - Reusability – function can be reused with different input values.
 - Efficiency – avoid repeating of codes

Functions with parameters

- 3 types of functions with parameters:
 - Pass by Value
 - Pass by Reference
 - Pass by Address

Functions – Pass by Value

- A **copy of the data is created** and **placed in a local variable** in the called function.
- Regardless of how the data are manipulated and changed in the called function, the **original data** in the calling function are **safe and remain unchanged**.
- This is the preferred passing technique for data protection.

Functions – Pass by Value

```
#include <iostream>
using namespace std;
void fun ( int x);
```

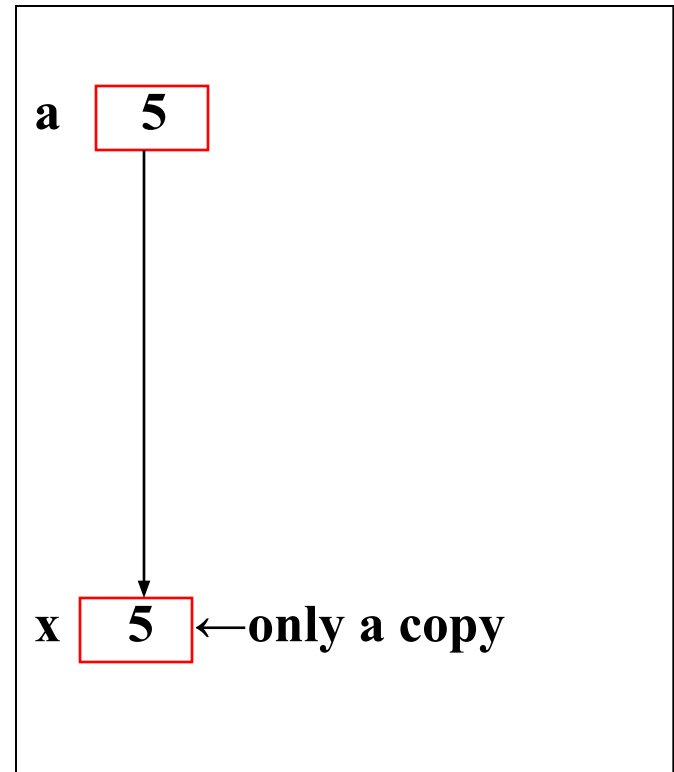
```
int main (void)
{
    int a=5;

    fun (a);
    cout << a;

    return 0;
}
```

**prints 5 (not
affected by the
function fun)**

```
void fun (int x) {
    x = x + 3;
}
```



Functions – Pass by Value

Exercise

- Write a function *checkMark* which will check a mark of a test. It will return true if the mark is in between 1 to 100 inclusive, otherwise return false.
- Write a main function to
 - ✓ prompt user to enter a mark of a test, then pass the mark to the *checkMark* function to check for its validity
 - ✓ display the result of the checking for the mark

Functions – Pass by Reference

- Links the variable identifiers in the calling function to their corresponding parameters in the called function.
- Any reference to a parameter is therefore the same as a reference to the variable in the calling function.
- When the called function changes a value, the variable in the calling function also changed
- Use a **reference variable (&)** as the parameter in the function, like creating an alias

Functions – Pass by Reference

```
# include <iostream>
using namespace std;
void fun ( int &x);
int main (void){
    int a=5;

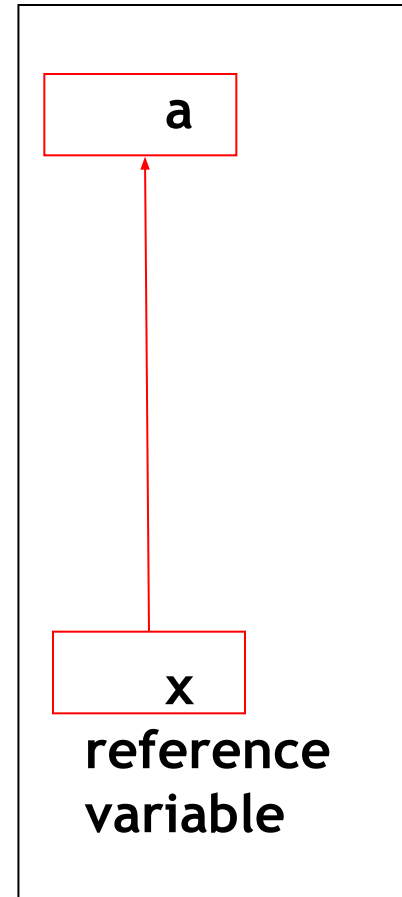
    fun (a);
    cout << a;
    return 0;
}
```

← no address operator '&'

← prints 8

```
void fun (int &x)
{
    x = x + 3;
}
```

← Does not
require '*'



Functions – Pass by Reference

Exercise 1

- A shop wants to update the price of an item during a sale. Write a function *applyDiscount* which receives *price* and *discount* to reduce the price of the item by the discount. Use pass by reference to update the original price.
- Write a main function to obtain the price and discount and invoke the *applyDiscount* function. Lastly it will display the discounted price.

Functions – Pass by Reference

Exercise 2

- Write a fitness function *updateSteps* with 3 reference parameters, allows users to update the daily walking steps (*todaySteps*) and also keep a running total of their steps for the week (*weeklySteps*) after adding the *extraSteps*.
- Write the *main* function to prompt users to enter extra steps after a morning walk at a park and then display *todaySteps* and *weeklySteps*. (Assume *todaySteps* and *weeklySteps* as 4700 and 21350 respectively.

Functions – Pass by Address

- Passing the **memory address (pointer)** of a variable to a function. Instead of copying the value (pass by value) or creating an alias (pass by reference)
- Function uses a **pointer** to the original data, so it can **access and modify it directly** using the address.
- **&** (address operator): used to pass an address
- ***** (indirection operator): used to store data at / to get data from an address

Functions – Pass by Address

```
# include <iostream>
using namespace std;
void fun ( int *x);
int main() {
    int a=5;

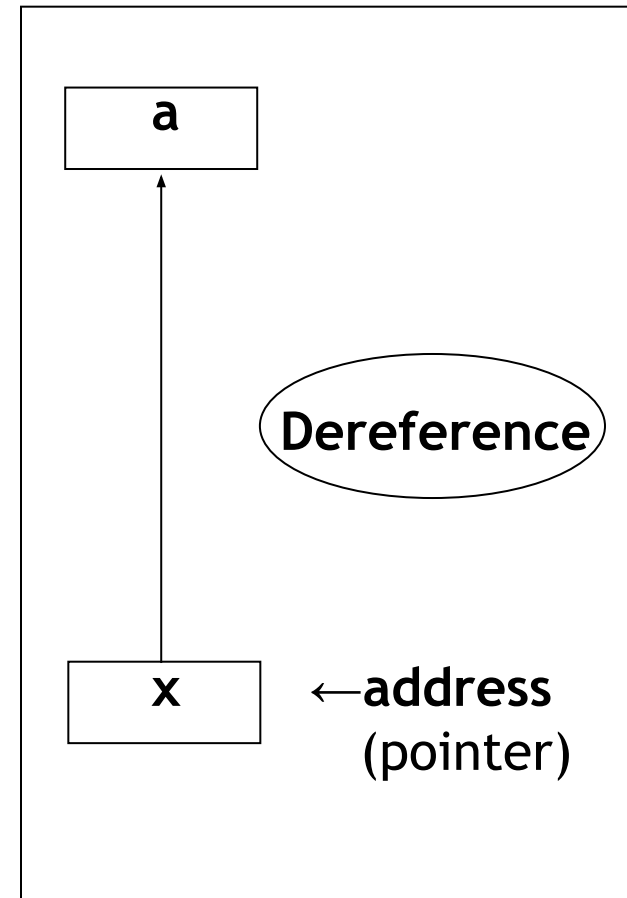
    fun (&a);
    cout << a;
    return 0;
}
```

← & : address operator

← prints 8

```
void fun (int *x)
{
    *x = *x + 3;
}
```

← Requires '*'
to dereference



Functions – Pass by Address

Exercise 1

- You are developing a mobile e-wallet app. When a user performs a top-up, the wallet balance will be updated with the amount required. Write the function *topUpWallet* with *topUpAmount* and *balance* parameters to perform the top up. Use pass-by-address to update the original balance.
- Write a main function to initialize *balance* = RM3.50, prompt the user for *topUpAmount* and invoke the *topUpWallet* function. Lastly it will display the updated balance.

Functions – Pass by Address

Exercise 2

- Each time a user withdraws from an ATM, the system will deduct the withdrawal amount from the balance and update the transaction count of the day of the user.
- Write a *withdrawCash* function with 3 parameters, *balance*, *transCount* and *withdrawAmt*. The function uses pass by address method.
- Write a main function to initialize account *balance* to RM4000, prompt the user for *withdrawAmt*, invoke the *withdrawCash* function and display the updated account balance.

Exchanging Two Variables using Pass by Reference functions

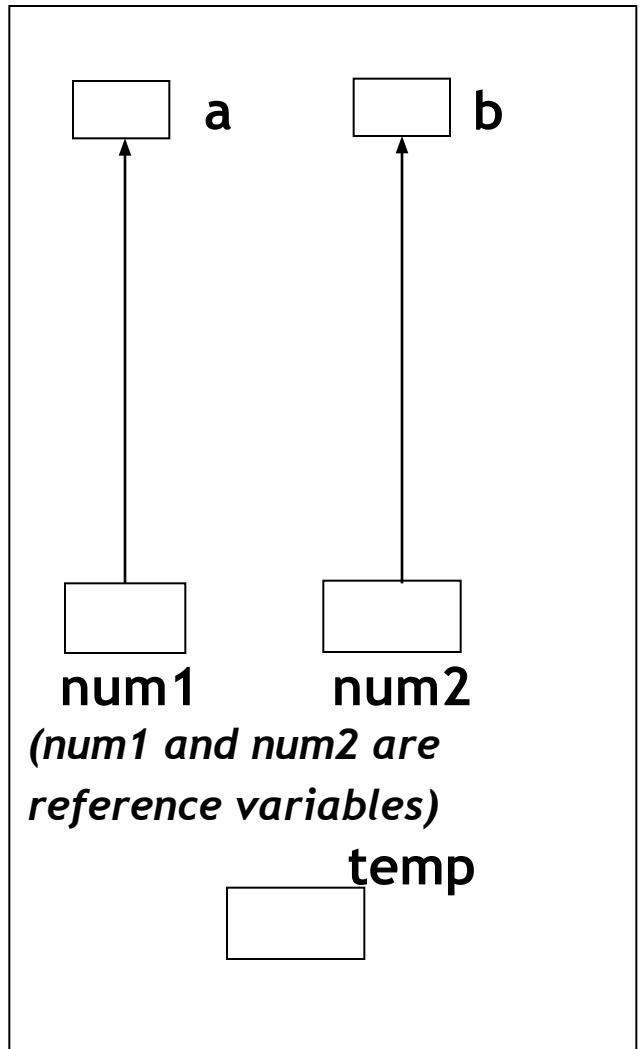
```
void exchange (int &num1,int &num2);
int main (void) {
    int a, b;
    .....
    exchange (a, b);
    .....
    return 0;
}
```

↑
type includes an ampersand

↑
No address operators

```
void exchange (int &num1, int &num2)
{
    int temp;
    temp = num1;
    num1 = num2;
    num2 = temp;
}
```

← no '*' is required



Exchanging Two Variables using Pass by Address functions

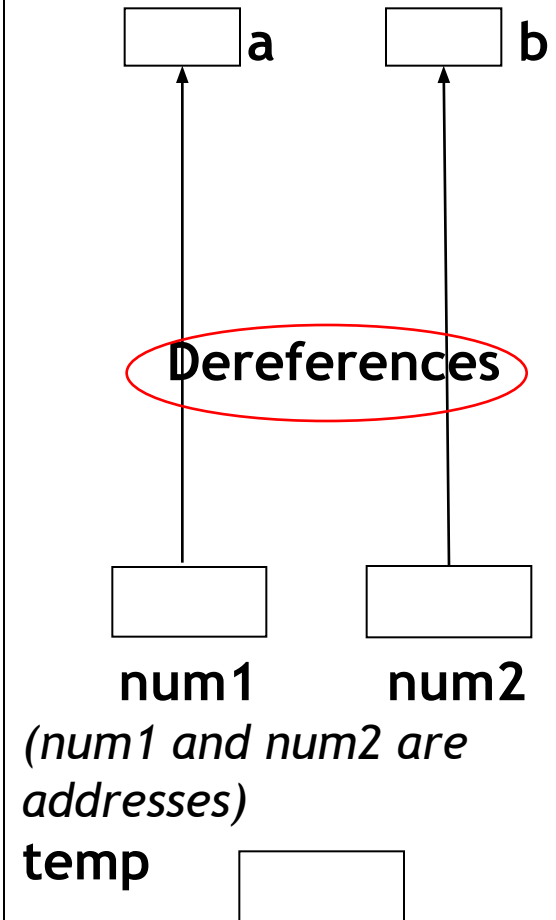
```
void exchange ( int *num1, int *num2);  
int main (void)  
{  
    int a, b;  
    .....  
    exchange (&a, &b);  
    .....  
    return 0;  
}
```

↑
type includes an asterisk

↑
address
operators

```
void exchange (int *num1, int *num2) {  
    int temp;  
    temp = *num1;  
    *num1 = *num2;  
    *num2 = temp;  
}
```

← Indirection
operator '*' is used
for dereferencing.



Functions – Pass by Reference & Address

Exercise

- You're building a fuel cost estimator for drivers. Write a *calculateFuelCost* function where it
 - receives distance (km), fuel rate (liters/km), and price per liter by **pass by value**
 - updates the fuel cost by **pass-by-reference**.
 - increases a trip counter by **pass-by-address**
- Write a *main* function where user enters total kilometers they will drive, the fuel consumption rate, and the price per liter of petrol. Then displays the fuel cost and number of trips.

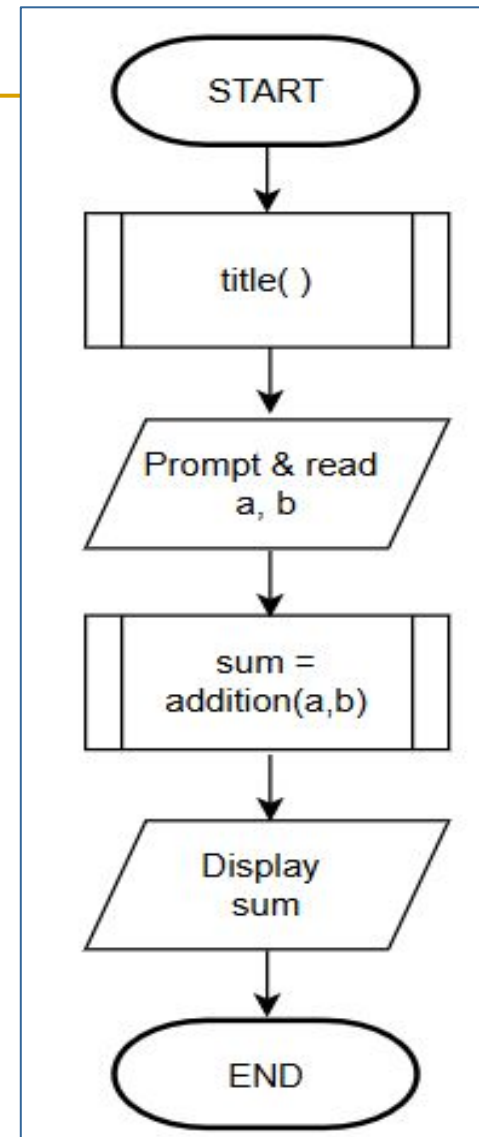
Flowchart & Pseudocode for Functions

```
#include <iostream>
using namespace std;
void title();
int addition(int, int);

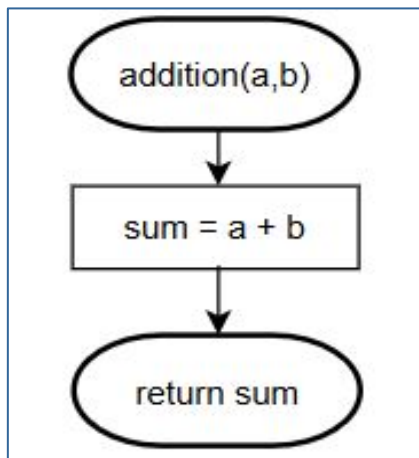
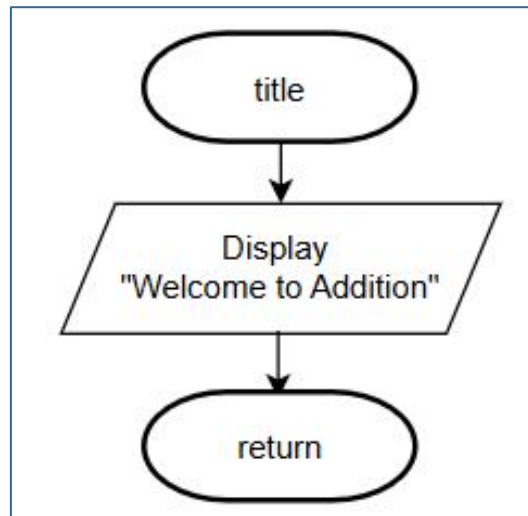
int main() {
    int sum, a, b;
    title();
    cout << "Enter 2 integers: ";
    cin >> a >> b;
    sum = addition(a,b);
    cout << "The result is " << sum << endl;
    return 0;
}

void title() {
    cout << "Welcome to Addition\n";
}

int addition(int a, int b) {
    return (a+b);
}
```



Flowchart & Pseudocode for Functions



START

Call `title()`

Prompt to enter 2 integers, `a` & `b`

`Sum = Call addition(a, b)`

Display `Sum`

END

`addition(a,b)`

`sum = a + b;`

Return `sum`

`title()`

Display "Welcome to Addition"

Return